

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 6

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Лавров Р. Д.

Преподаватель Цалкович П. А.

Москва 2026

1 Задача

1. Базовой лабораторной работой является лабораторная работа №3 (модельно-видовые преобразования и преобразования проецирования).

2. Определить параметры модели освещения OpenGL (свойства источника света, свойства материалов (поверхностей), характеристики глобальной модели освещения).

3. Исследовать один из методов повышения реалистичности получаемых изображений сцены (в соответствии с вариантом): А1. использование сфокусированного источника света;

4. Реализовать один из алгоритмов анимации (в соответствии с вариантом): Б2. моделирование движения тела (с заданной начальной скоростью) при условии абсолютно упругого отражения объекта от границ некоторого ограничивающего объема (регулярной формы);

5. Реализовать наложение текстуры (загрузка из файла *.bmp или процедурная генерация) с возможностью отключения (в соответствии с вариантом): В2. использование текстуры для определения свойств поверхности (модулирование коэффициента диффузного отражения);

2 Основная теория

2.1 Модель освещения в OpenGL

В компьютерной графике для расчёта освещённости поверхности обычно используется модель Фонга, которая учитывает три составляющие: фоновую (ambient), диффузную (diffuse) и зеркальную (specular). В OpenGL эта модель реализована через параметры источников света, материалов и глобальные настройки сцены.

Интенсивность света в точке поверхности вычисляется по формуле (точечный источник):

$$I = I_a k_a + \frac{I_l}{k_c + k_l d + k_q d^2}\big(k_d(\vec{n} \cdot \vec{L}) + k_s(\vec{R} \cdot \vec{V})^n\big),$$

где I_a – интенсивность глобального фонового света, k_a, k_d, k_s – коэффициенты отражения материала для фоновой, диффузной и зеркальной составляющих соответственно, I_l – интенсивность источника, d – расстояние от источника до точки, k_c, k_l, k_q – коэффициенты затухания, $\vec{n}$ – нормаль, $\vec{L}$ – направление на источник, $\vec{V}$ – направление на наблюдателя, $\vec{R}$ – вектор зеркального отражения, n – коэффициент блеска.

В OpenGL свойства источника задаются функциями $\mathrm{glLightfv}$ с параметрами $\mathrm{GL_AMBIENT}$, $\mathrm{GL_DIFFUSE}$, $\mathrm{GL_SPECULAR}$, $\mathrm{GL_POSITION}$. Глобальная фоновой подсветка устанавливается через $\mathrm{glLightModelfv(GL_LIGHT_MODEL_AMBIENT, ...)}$. Материал поверхности определяется с помощью $\mathrm{glMaterialfv}$ или, для упрощения, включается режим $\mathrm{glColorMaterial}$, который связывает цвет вершины с диффузной и фоновой компонентами материала.

2.2 Сфокусированный источник света

Направленный сфокусированный источник (прожектор) излучает свет внутри конуса с вершиной в точке источника. Его действие описывается дополнительными параметрами:

- $\mathrm{GL_SPOT_CUTOFF}$ – половинный угол раствора конуса (в градусах); если угол между направлением на точку и осью конуса превышает это значение, точка не освещается данным прожектором.
- $\mathrm{GL_SPOT_DIRECTION}$ – вектор направления оси конуса.
- $\mathrm{GL_SPOT_EXPONENT}$ – показатель степени концентрации света в центре пятна. Интенсивность внутри конуса дополнительно модулируется множителем $(\cos\beta)^{\varepsilon}$, где β – угол отклонения от оси, ε – значение экспоненты. Чем выше ε, тем более резко спадает яркость от центра к краям.

2.3 Анимация с учётом физических законов

Анимация объекта, движущегося с заданной начальной скоростью и испытывающего абсолютно упругие столкновения с границами ограничивающего объёма (например, прямоугольного параллелепипеда), требует учёта физических законов. Положение объекта в каждый момент времени t определяется через скорость $\vec{v}$ и ускорение $\vec{a}$:

$$\vec{p}(t) = \vec{p}_0 + \vec{v}_0 t + \frac{1}{2}\vec{a}t^2.$$

В простейшем случае ускорение отсутствует, и на каждом шаге моделирования выполняется сдвиг $\vec{p} \leftarrow \vec{p} + \vec{v} \cdot \Delta t$, где Δt – интервал времени между кадрами.

Для обнаружения столкновения сферы радиуса r с плоской границей прямоугольного объёма, заданного интервалами $[x_{\min}, x_{\max}]$, $[y_{\min}, y_{\max}]$, $[z_{\min}, z_{\max}]$, используется условие выхода центра сферы за пределы допуска: $p_i > \text{bound}_i$ или $p_i < -\text{bound}_i$. При абсолютно упругом ударе о горизонтальную или вертикальную плоскость угол падения равен углу отражения, а модуль скорости не изменяется. Для граней, параллельных координатным плоскостям, это означает простое изменение знака соответствующей компоненты скорости: $v_i \leftarrow -v_i$.

2.4 Наложение текстуры и модуляция диффузного отражения

Текстурирование позволяет повысить детализацию поверхности без усложнения геометрии. В OpenGL текстура загружается функцией $\mathrm{glTexImage2D}$ и привязывается к объекту с помощью текстурных координат (s, t), задаваемых для каждой вершины (через $\mathrm{glTexCoord2f}$). Для сферы удобно использовать сферическое отображение:

$$s = \frac{\text{atan2}(z, x)}{2\pi}, \qquad t = \frac{(y/R) + 1}{2},$$

где R – радиус сферы, y – вертикальная координата точки.

Режим применения текстуры определяется функцией $\mathrm{glTexEnv}$ с параметром $\mathrm{GL_TEXTURE_ENV_MODE}$. Значение $\mathrm{GL_MODULATE}$ предписывает

умножать цвет текстуры на текущий цвет вершины. Поскольку в работе включено glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE), цвет вершины задаёт диффузный коэффициент отражения k_d. Таким образом, текстура модулирует этот коэффициент по поверхности, создавая эффект неоднородного материала: светлые участки текстуры соответствуют большему k_d и, как следствие, более яркому диффузному отражению, а тёмные – меньшему.

3 Практическая реализация

Листинг 1: Основной код

```
import glfw
from PIL import Image
from OpenGL.GL import *
import numpy as np
import math
import time

angle_x, angle_y = 0, 0
wireframe = False
scale  = 0.8
texture_enabled = True
texture_id = None

pos = [-0.3,  0.0,  0.0]
vel = [0.5,  0.4,  0.3]
BOUND = 1.1
SPHERE_RADIUS = 1.0
last_time = time.time()

def three_point_projection():
    return [
        0.87,  -0.09,  0.98,  0.49,
        0,      0.98,  0.35,  0.17,
        0.5,    0.15,-1.7,   -0.85,
        0,      0,     1,     2
    ]

def setup_lighting():
    glEnable(GL_LIGHTING)
    glEnable(GL_LIGHT0)

    glLightfv(GL_LIGHT0, GL_AMBIENT, [0.3, 0.3, 0.3, 1])
```

```
    glLightfv(GL_LIGHT0, GL_DIFFUSE, [3, 3, 3, 1])
    glLightfv(GL_LIGHT0, GL_SPECULAR, [3, 3, 3, 1])
    glLightfv(GL_LIGHT0, GL_POSITION, [0, 3, 0, 1])

    glLightf(GL_LIGHT0, GL_SPOT_CUTOFF, 70)
    glLightfv(GL_LIGHT0, GL_SPOT_DIRECTION, [0, -1, 0])
    glLightf(GL_LIGHT0, GL_SPOT_EXPONENT, 70)

    glLightModelfv(GL_LIGHT_MODEL_AMBIENT,  [0.3, 0.3, 0.3, 1.0])
    glLightModeli(GL_LIGHT_MODEL_LOCAL_VIEWER, GL_FALSE)
    glLightModeli(GL_LIGHT_MODEL_TWO_SIDE, GL_FALSE)

def setup_material():
    glEnable(GL_COLOR_MATERIAL)
    glColorMaterial(GL_FRONT, GL_AMBIENT_AND_DIFFUSE)
    glMaterialfv(GL_FRONT, GL_SPECULAR, [0.8, 0.8, 0.8, 1.0])
    glMaterialf(GL_FRONT, GL_SHININESS, 64.0)

def create_checkerboard_texture(size=64):
    img = Image.open('3.jpg').resize((128,  128))
    img_data = img.tobytes()
    width, height = img.size
    texture = glGenTextures(1)
    glBindTexture(GL_TEXTURE_2D, texture)
    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGB,width, height, 0,GL_RGB,
     GL_UNSIGNED_BYTE, img_data)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT)
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT)
    return texture

def draw_part(y_bottom, R1, R2, height, sides, R):
    angle_step = 2 * math.pi / sides
    angle = 0.0
    glBegin(GL_QUAD_STRIP)
    for _ in range(sides + 1):
        x1 = R1 * math.cos(angle)
        z1 = R1 * math.sin(angle)
        x2 = R2 * math.cos(angle)
        z2 = R2 * math.sin(angle)
        y_top = y_bottom + height

        if R1 > 0.001:
            nx1, ny1, nz1 = x1/R, y_bottom/R, z1/R
        else:
```

```
78              nx1, ny1, nz1 = 0.0,  -1.0,  0.0
79          if R2 > 0.001:
80              nx2, ny2, nz2 = x2/R, y_top/R, z2/R
81          else:
82              nx2, ny2, nz2 = 0.0,  1.0,  0.0
83          u = angle / (2 * math.pi)
84          v1 = (y_bottom / R + 1.0) / 2.0
85          v2 = (y_top / R + 1.0) / 2.0
86
87          glNormal3f(nx1, ny1, nz1)
88          glTexCoord2f(u, v1)
89          glVertex3f(x1,  y_bottom, z1)
90
91          glNormal3f(nx2, ny2, nz2)
92          glTexCoord2f(u, v2)
93          glVertex3f(x2,  y_top, z2)
94
95          angle += angle_step
96      glEnd()
97
98  def draw_sphere():
99      R = SPHERE_RADIUS
100     segments = 10
101     glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else GL_FILL)
102     for i in range(segments):
103         theta1 = -math.pi/2 + (i / segments) * math.pi
104         theta2 = -math.pi/2 + ((i + 1) / segments) * math.pi
105         y1 = R * math.sin(theta1)
106         y2 = R * math.sin(theta2)
107         r1 = R * math.cos(theta1)
108         r2 = R * math.cos(theta2)
109         height = y2 - y1
110         if r1 >= 0 and r2 >= 0:
111             draw_part(y1, r1, r2, height, 20, R)
112
113 def display():
114     global angle_x, angle_y, scale, pos, vel, last_time, texture_enabled, texture_id
115     current_time = time.time()
116     dt = current_time - last_time
117     last_time = current_time
118     if dt > 0.1:
119         dt = 0.016
120     for i in range(3):
121         pos[i] += vel[i] * dt
122         if pos[i] > BOUND:
123             pos[i] = BOUND
```

```
124                 vel[i] = -vel[i]
125             elif pos[i] < -BOUND:
126                 pos[i] = -BOUND
127                 vel[i] = -vel[i]
128 
129         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
130 
131         glMatrixMode(GL_PROJECTION)
132         glLoadIdentity()
133         glMultMatrixf(three_point_projection())
134         glMatrixMode(GL_MODELVIEW)
135         glLoadIdentity()
136         glEnable(GL_NORMALIZE)
137 
138         if texture_enabled:
139             glEnable(GL_TEXTURE_2D)
140             glBindTexture(GL_TEXTURE_2D, texture_id)
141             glTexEnvi(GL_TEXTURE_ENV, GL_TEXTURE_ENV_MODE, GL_MODULATE)
142         else:
143             glDisable(GL_TEXTURE_2D)
144 
145         glTranslatef(pos[0], pos[1], pos[2])
146         glScalef(scale, scale, scale)
147         glRotatef(angle_x, 1, 0, 0)
148         glRotatef(angle_y, 0, 1, 0)
149         glColor3f(1, 1, 1)
150         draw_sphere()
151 
152         glDisable(GL_TEXTURE_2D)
153 
154 def key_callback(window, key, scancode, action, mods):
155         global angle_x, angle_y, wireframe, scale, texture_enabled
156         if action in [glfw.PRESS, glfw.REPEAT]:
157             if key == glfw.KEY_RIGHT:
158                 angle_y -= 3
159             if key == glfw.KEY_LEFT:
160                 angle_y += 3
161             if key == glfw.KEY_UP:
162                 angle_x -= 3
163             if key == glfw.KEY_DOWN:
164                 angle_x += 3
165             if key == glfw.KEY_KP_ADD or key == glfw.KEY_EQUAL:
166                 scale += 0.1
167             if key == glfw.KEY_KP_SUBTRACT or key == glfw.KEY_MINUS:
168                 scale = max(0.1, scale - 0.1)
169             if key == glfw.KEY_M:
```

```
170                 wireframe = not wireframe
171             if key == glfw.KEY_T:
172                 texture_enabled = not texture_enabled
173 
174 def main():
175     global texture_id, last_time
176     if not glfw.init():
177         return
178     window = glfw.create_window(800, 800, "lab6", None, None)
179     if not window:
180         glfw.terminate()
181         return
182     glfw.make_context_current(window)
183     glfw.set_key_callback(window, key_callback)
184 
185     glEnable(GL_DEPTH_TEST)
186     glClearColor(0.15, 0.15, 0.15, 1.0)
187     setup_lighting()
188     setup_material()
189     texture_id = create_checkerboard_texture()
190     last_time = time.time()
191     while not glfw.window_should_close(window):
192         display()
193         glfw.swap_buffers(window)
194         glfw.poll_events()
195     glfw.destroy_window(window)
196     glfw.terminate()
197 
198 if __name__ == "__main__":
199     main()
```

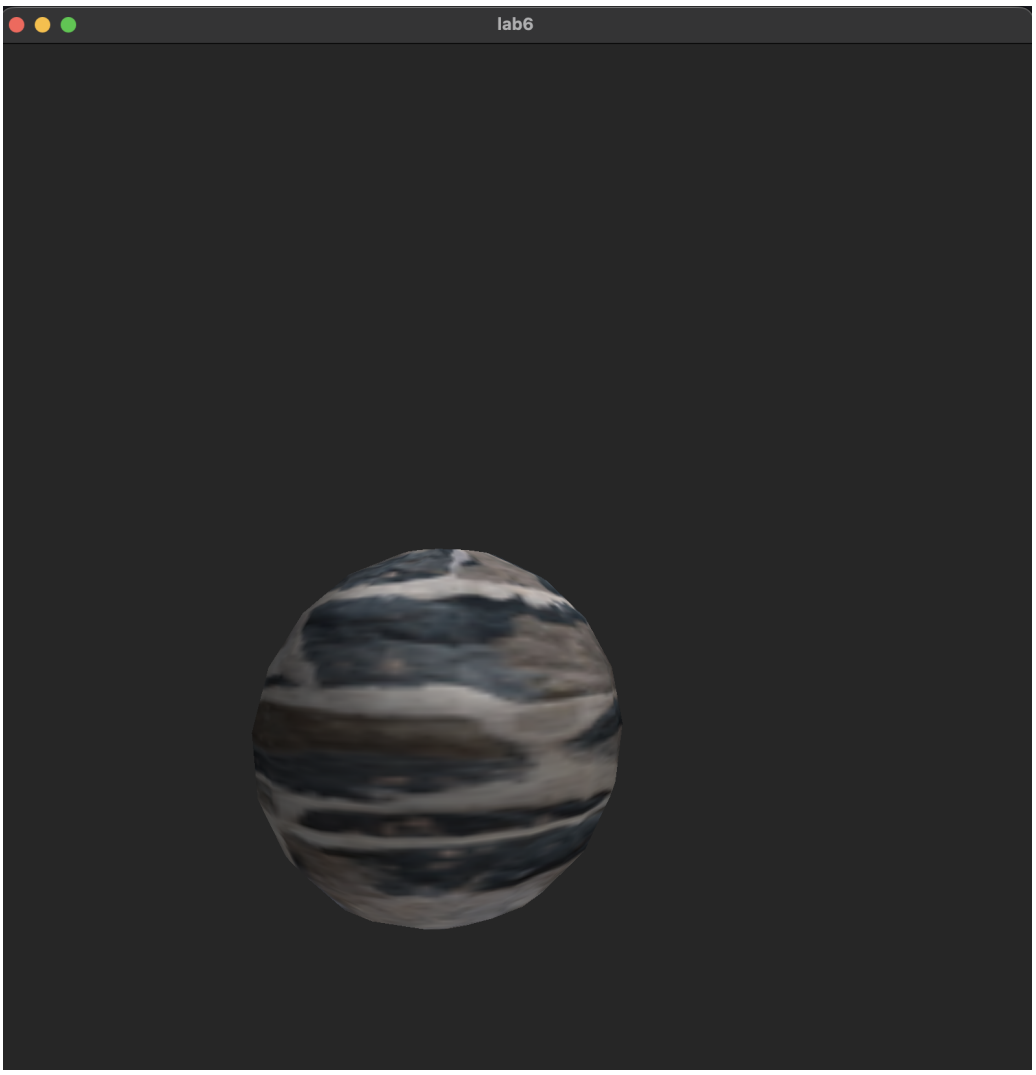


Рис. 1 — Результат работы

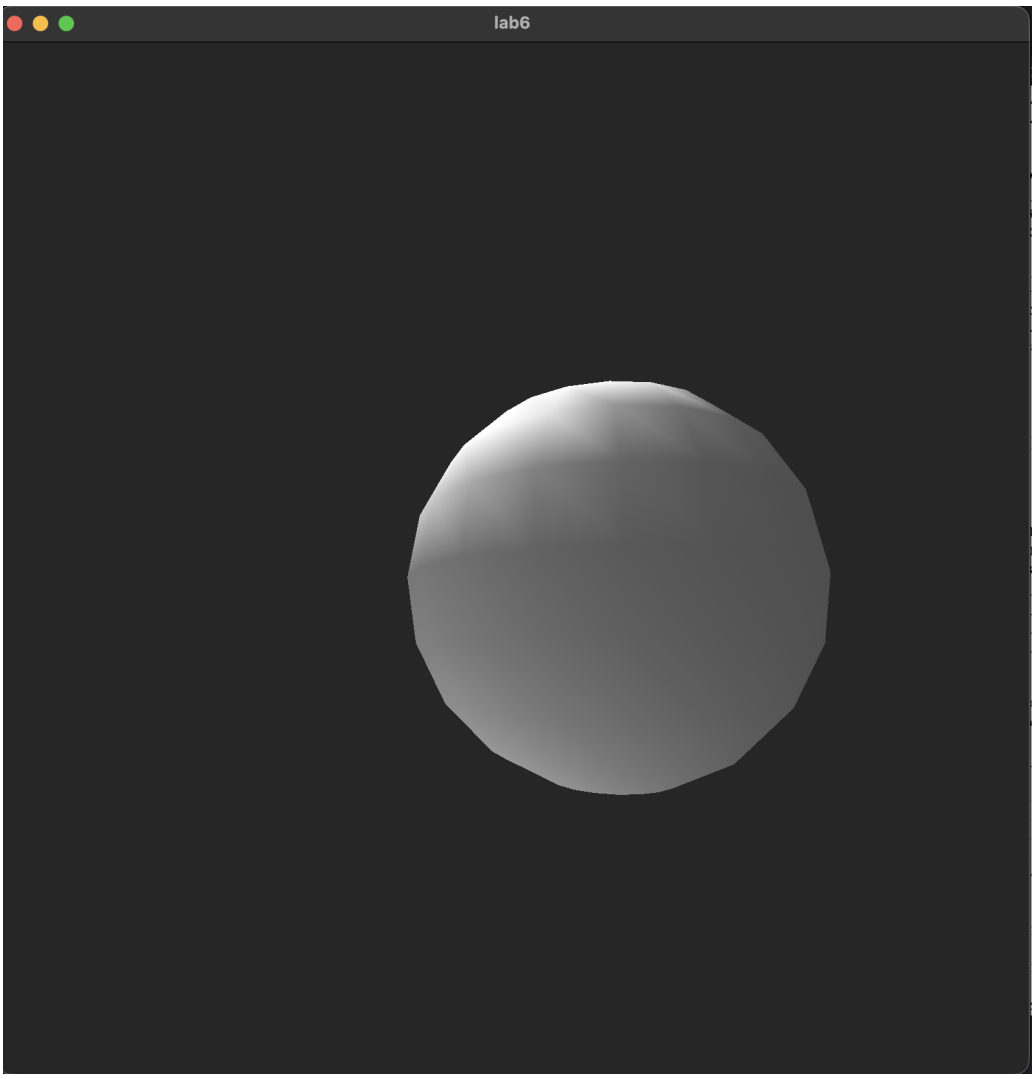


Рис. 2 — Результат работы

4 Вывод

В работе настроена модель освещения OpenGL с одним прожекторным источником, заданы параметры материала и глобальная фоновая подсветка. Реализована анимация движения сферы внутри ограничивающего куба с абсолютно упругим отражением от границ. Выполнено наложение текстуры, модулирующей коэффициент диффузного отражения (по варианту B2), с возможностью её отключения.